

Diagrama 1 - Arquitectura de software (driver C + libstats escalar / AVX2)

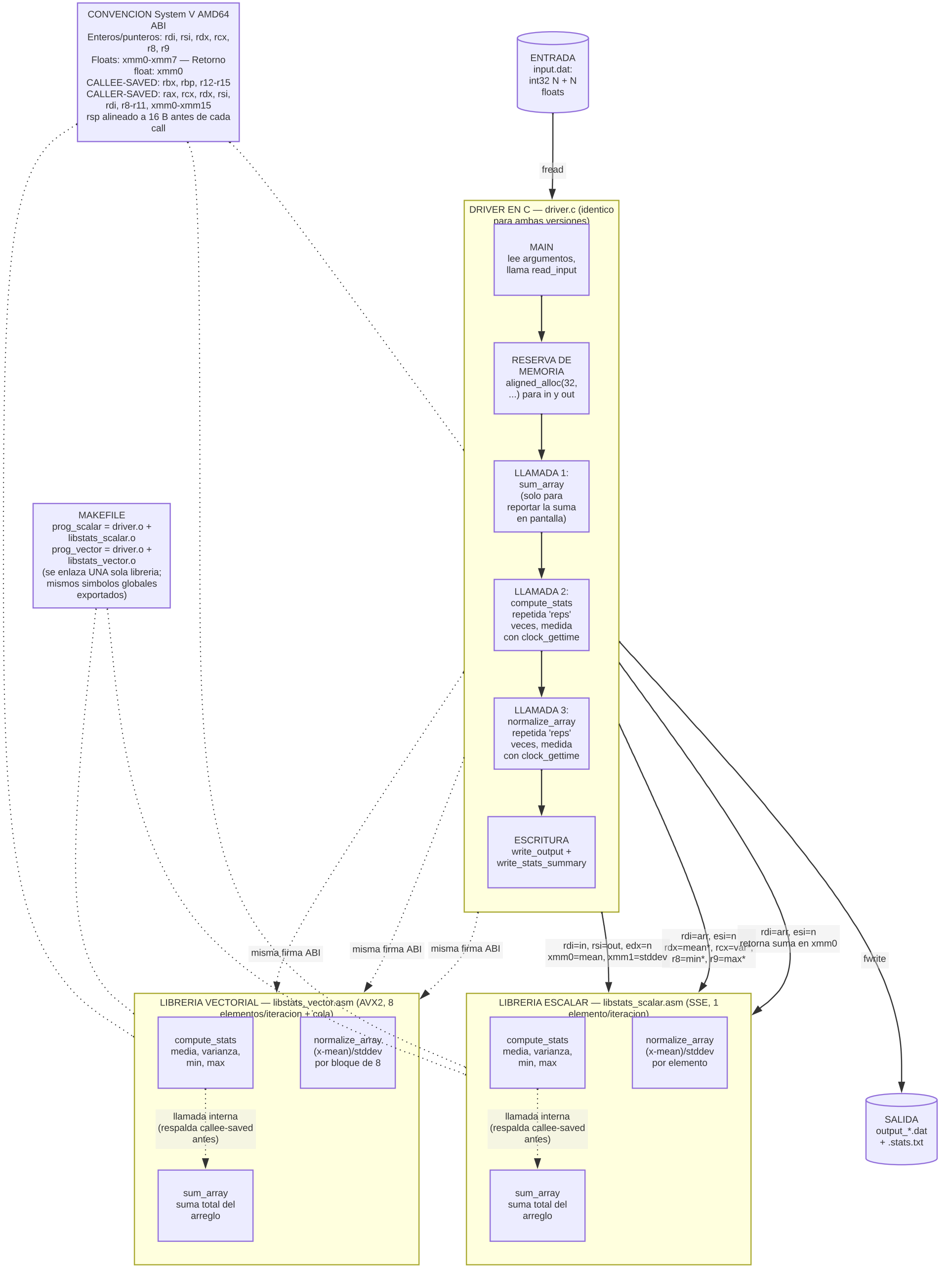


Diagrama 2 - Flujo de control del bucle principal (escalar vs AVX2)

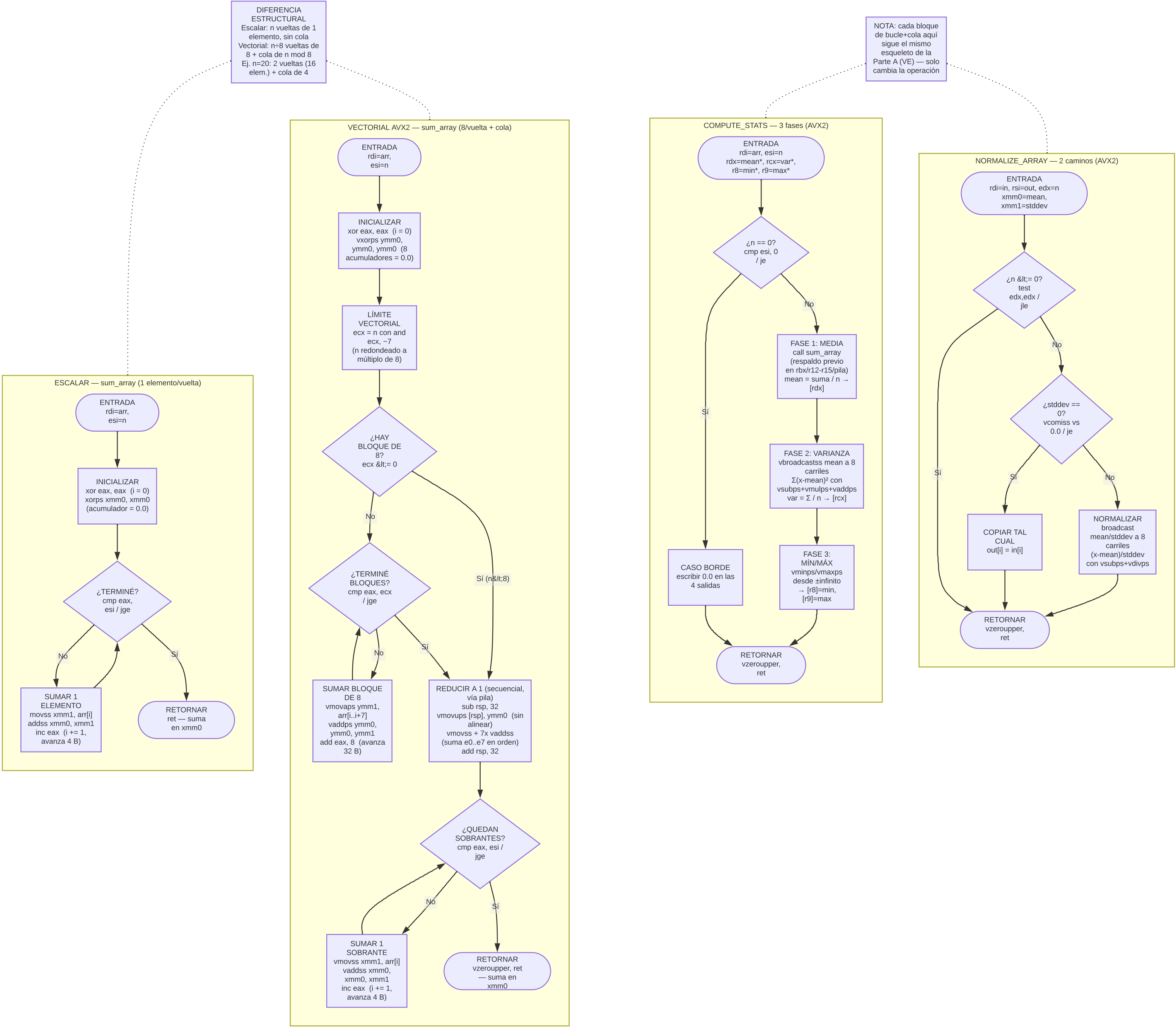


Diagrama 3 - Qué guarda cada registro en cada etapa (kernel vectorial AVX2, con la versión escalar como referencia)

ETAPA 1 - Entrada a compute_stats (si n = 0 escribe 0.0 en las 4 salidas y termina)		
Registro	En palabras simples	Contenido concreto
rdi	Recibo el arreglo de entrada	puntero arr (alineado a 32 B)
esi	Recibo el tamaño	n = cantidad de floats
rdx	Recibo dónde dejar la media	puntero mean*
rcx	Recibo dónde dejar la varianza	puntero var*
r8	Recibo dónde dejar el mínimo	puntero min*
r9	Recibo dónde dejar el máximo	puntero max*

Registro	En palabras simples	Contenido concreto
rbx	Guardo dónde va la media	copia de mean* (callee-saved)
r12	Guardo dónde va la varianza	copia de var*
r13	Guardo dónde va el mínimo	copia de min*
r14	Guardo dónde va el máximo	copia de max*
r15	Guardo el arreglo de entrada	copia de arr
pila	Guardo n (dos veces)	2 x push rsi: deja rsp alineado a 16 B

ETAPA 3 - Media: sum_array + división entre n (8 floats por vuelta)		
Registro	En palabras simples	Contenido concreto
rdi / esi	Recibo arreglo y tamaño	arr y n, igual que en la ABI
eax	Llevo el índice i	0, 8, 16, ... (sube de 8 en 8 = 32 B)
ecx	Guardo el límite vectorial	n redondeado hacia abajo a múltiplo de 8
ymm0	Guardo el acumulador	8 sumas parciales, una por carril
ymm1	Guardo el bloque actual	8 floats cargados con vmovaps
rsp	Reservo espacio temporal en la pila para la reducción	sub rsp, 32 Reservo 32 bytes para hacer la reducción, luego se devuelven con add rsp, 32
xmm0	Dejo la suma total	Tras sumar secuencialmente (vmovss + 7×vaddss) los 8 floats, el resultado queda aquí
xmm1 / xmm0	Calculo la media	xmm1 = n como float; xmm0 = suma / n, se guarda en [rdx]

Registro	En palabras simples	Contenido concreto
eax	Llevo el índice i	0, 8, 16, ... (de 8 en 8)
r10d	Guardo el límite vectorial	n redondeado hacia abajo a múltiplo de 8 (Recalculado)
ymm0	Guardo el acumulador	8 sumas parciales de (x - media)^2 (Re-inicializado)
ymm1	Guardo la media repetida en los 8 carriles	vbroadcastss desde [rdx]
ymm2	Guardo el bloque de trabajo	x, luego x - media, luego su cuadrado
rsp	Reservo espacio temporal en la pila para la reducción	sub rsp, 32 Reservo 32 bytes para hacer la reducción, luego se devuelven con add rsp, 32
xmm1	Guardo n convertido a float	vcvtss2ss desde esi
xmm0	Dejo la varianza	suma total / n, se guarda en [rcx]

ETAPA 5 - Mínimo y máximo con vminps / vmaxps (8 floats por vuelta)		
Registro	En palabras simples	Contenido concreto
eax	Llevo el índice i	0, 8, 16, ... (de 8 en 8)
r10d	Guardo el límite vectorial	n redondeado hacia abajo a múltiplo de 8
ymm0	Guardo los mínimos parciales	8 carriles, empiezan en +infinito
ymm1	Guardo los máximos parciales	8 carriles, empiezan en -infinito
ymm2	Guardo el bloque actual	8 floats de arr
xmm2 / xmm3	Guardo temporales para reducir	mitad alta (vextractf128) y carriles intercambiados (vshufps)
xmm0	Dejo el mínimo final	carril 0, se guarda en [r8]
xmm1	Dejo el máximo final	carril 0, se guarda en [r9]

Registro	En palabras simples	Contenido concreto
rdi / rsi	Recibo entrada y salida	punteros in y out (alineados a 32 B)
edx	Recibo el tamaño	n
xmm0 / xmm1	Recibo la media y la desviación	mean y stddev (luego se pisan con el broadcast)
xmm8 / xmm9	Guardo copias de media y desviación	para comparar stddev con 0 y para la cola escalar
ymm0	Guardo la media en los 8 carriles	vbroadcastss desde xmm0
ymm1	Guardo la desviación en los 8 carriles	vbroadcastss desde xmm1
ymm2	Guardo el bloque de trabajo	in, luego in - media, luego dividido = out (vmovaps)
eax / r10d	Llevo el índice y el límite vectorial	i sube de 8 en 8; límite = n redondeado a múltiplo de 8
xmm3 / xmm2	Guardo el sobrante y su resultado (cola)	xmm3 = in[i]; xmm2 = (in[i] - media) / desviación

REFERENCIA - Versión escalar (SSE): mismos roles, un solo float por vuelta

Registro	En palabras simples	Contenido concreto
eax	Llevo el índice i	0, 1, 2, ... (sube de 1 en 1 = 4 B)
xmm0	Guardo el acumulador	una sola suma (en min/max guarda el mínimo)
xmm1	Guardo el elemento de trabajo	arr[i] o (arr[i] - media)^2 (en min/max guarda el máximo)
xmm2	Guardo el elemento a comparar	arr[i] en el bucle de min/max
xmm8 / xmm9	Guardo media y desviación	copias usadas en normalize_array
rbx, r12-r15	Respaldo de punteros	igual que en la vectorial; la media se relee de [rbx]

Cómo leer las tablas

ymm = 8 floats a la vez (256 bits); cada posición es un carril (32 bits).
xmm = los 4 floats bajos de un ymm, o un solo float en código escalar.
Callee-saved (hay que preservarlos): rbx, rbp, r12-r15. El resto son caller-saved.
Límite vectorial = n redondeado hacia abajo a múltiplo de 8; los n mod 8 sobrantes van a la cola escalar.
vzeroupper antes de cada ret limpia la mitad alta de los ymm.

Diagrama 4 - Memoria: arreglos in[] y out[], alineación y avance del puntero (escalar vs AVX2, ejemplo n = 20)

